

Conversion of NFA to DFA, Minimization of DFA

Birzhan Kalmurzayev

Ac. year 2025-2026

Clearly, NFA automata are difficult to implement in reality. Therefore, the problem arises of converting NFA automata into DFA, that is, a DFA automaton that accepts the same language as NFA.

To see this conversion, let's consider an NFA $M = (Q, \Sigma, \delta, q_0, F)$. For any $x \in \Sigma^*$, let Q_x be the set of states which are end states of the computation paths of x in M , that is $Q_x = \delta^*(q_0, x)$. In particular, $Q_\epsilon = \epsilon\text{-closure}(\{q_0\}) = \{q \in Q : \delta(q_0, \epsilon) = q\}$. So, $x \in \Sigma^*$ is accepted by M iff $Q_x \cap F \neq \emptyset$. Thus, it suggests that we can use these subsets $Q_x \subset Q$ as the states for equivalent DFA.

Formally, we would like construct a DFA

$M' = (Q', \Sigma, \delta', Q_\varepsilon, F')$, where

- $Q' = \{Q_x : x \in \Sigma^*\};$
- $F' = \{Q_x : Q_x \cup F \neq \emptyset\};$
- $\delta'(Q_x, a) = Q_{xa},$ for $x \in \Sigma^*$ and $a \in \Sigma.$

Formally, we would like construct a DFA

$M' = (Q', \Sigma, \delta', Q_\varepsilon, F')$, where

- $Q' = \{Q_x : x \in \Sigma^*\};$
- $F' = \{Q_x : Q_x \cup F \neq \emptyset\};$
- $\delta'(Q_x, a) = Q_{xa},$ for $x \in \Sigma^*$ and $a \in \Sigma.$

Note that each Q_x is a subset of Q , and so Q' is a finite set.

Formally, we would like construct a DFA

$M' = (Q', \Sigma, \delta', Q_\varepsilon, F')$, where

- $Q' = \{Q_x : x \in \Sigma^*\};$
- $F' = \{Q_x : Q_x \cup F \neq \emptyset\};$
- $\delta'(Q_x, a) = Q_{xa},$ for $x \in \Sigma^*$ and $a \in \Sigma.$

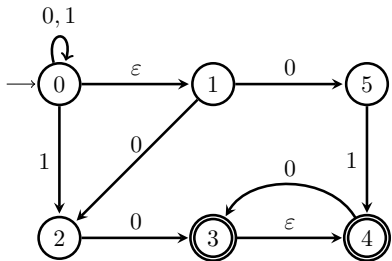
Note that each Q_x is a subset of Q , and so Q' is a finite set.

Theorem

A language is accepted by an NFA iff it is accepted by DFA.

Example

Find a DFA which is equivalent to the following NFA:



Find an DFA for the following languages (Hint: First find NFA and convert it to DFA)

- 1 all binary strings in which the third symbol from right is 0;

Find an DFA for the following languages (Hint: First find NFA and convert it to DFA)

- 1 all binary strings in which the third symbol from right is 0;
- 2 all binary strings that contain both substring 010 and 101;

Find an DFA for the following languages (Hint: First find NFA and convert it to DFA)

- 1 all binary strings in which the third symbol from right is 0;
- 2 all binary strings that contain both substring 010 and 101;
- 3 all binary strings that contain both 001 and 110 as substring or contain neither 001 nor 110 as a substring.

Find an DFA for the following languages (Hint: First find NFA and convert it to DFA)

- 1 all binary strings in which the third symbol from right is 0;
- 2 all binary strings that contain both substring 010 and 101;
- 3 all binary strings that contain both 001 and 110 as substring or contain neither 001 nor 110 as a substring.
- 4 binary strings which regular expression is $10 + (0 + 11)0^*1$;

Find an DFA for the following languages (Hint: First find NFA and convert it to DFA)

- 1 all binary strings in which the third symbol from right is 0;
- 2 all binary strings that contain both substring 010 and 101;
- 3 all binary strings that contain both 001 and 110 as substring or contain neither 001 nor 110 as a substring.
- 4 binary strings which regular expression is $10 + (0 + 11)0^*1$;
- 5 binary strings which regular expression is $0^*1(0 + 10^*1)^*$;

Find an DFA for the following languages (Hint: First find NFA and convert it to DFA)

- 1 all binary strings in which the third symbol from right is 0;
- 2 all binary strings that contain both substring 010 and 101;
- 3 all binary strings that contain both 001 and 110 as substring or contain neither 001 nor 110 as a substring.
- 4 binary strings which regular expression is $10 + (0 + 11)0^*1$;
- 5 binary strings which regular expression is $0^*1(0 + 10^*1)^*$;

Properties of regular languages

Theorem

The class of regular languages is closed under

- *union;*
- *intersection;*
- *subtraction;*
- *complementation;*
- *concatenation;*
- *Kleene closure;*
- *Reversal.*

A lot of the examples we looked at today when constructing DFA automata can have many (extra) states. Many DFA's constructed by the converting from NFA can be reduced to smaller, equivalent DFA's.

A lot of the examples we looked at today when constructing DFA automata can have many (extra) states. Many DFA's constructed by the converting from NFA can be reduced to smaller, equivalent DFA's.

So, lets discuss how to find, for a given regular language, a DFA with the minimum number of states. Such DFA is called **minimum DFA**.

To begin this, we define, for any regular language L , an equivalence relation R_L on Σ^* as follows:

$$x R_L y \iff \forall \omega [x\omega \in L \leftrightarrow y\omega \in L].$$

Recall that for any equivalence relation R on the set S and any $x \in S$, the class

$$[x]_R := \{y \in S : x R y\}$$

is called an equivalence class containing x .

To begin this, we define, for any regular language L , an equivalence relation R_L on Σ^* as follows:

$$x R_L y \iff \forall \omega [x\omega \in L \leftrightarrow y\omega \in L].$$

Recall that for any equivalence relation R on the set S and any $x \in S$, the class

$$[x]_R := \{y \in S : x R y\}$$

is called an equivalence class containing x .

Example. Let $L = \{x \in \{0,1\}^* : |x| \text{ is odd}\}$. Find all equivalence classes of R_L .

Lemma

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA which accepted a language L . Then for any $q \in Q$ if for words x and y $\delta^(q_0, x) = \delta^*(q_0, y)$, then $x R_L y$.*

Lemma

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA which accepted a language L . Then for any $q \in Q$ if for words x and y $\delta^(q_0, x) = \delta^*(q_0, y)$, then $x R_L y$.*

Theorem

For any regular language L , its minimum DFA has exactly $\text{Index}(R_L)$ states. Here $\text{Index}(R)$ denote the number of equivalence classes of R .

We can construct DFA $M = (Q, \Sigma, \delta, q_0, F)$ which accept regular language L as follows:

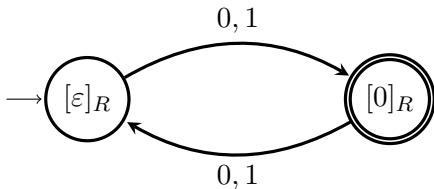
- $Q = \{[x]_{R_L} : x \in \Sigma^*\};$
- $\delta([x]_{R_L}, a) = [xa]_{R_L};$
- $q_0 = [\varepsilon]_{R_L};$
- $F = \{[x]_{R_L} : x \in L\}.$

Example.

For language $L = \{x \in \{0, 1\}^* : |x| \text{ is odd}\}$ construct minimal DFA.

Example.

For language $L = \{x \in \{0, 1\}^* : |x| \text{ is odd}\}$ construct minimal DFA.



In general, finding the equivalence classes of R_L for a given language L may not be easy. However, if we are given DFA M for L , then there is a simple method to find the equivalent minimum DFA.

In general, finding the equivalence classes of R_L for a given language L may not be easy. However, if we are given DFA M for L , then there is a simple method to find the equivalent minimum DFA.

The basic idea is the following: For any DFA M , two input strings which end at same state M must belong to the same equivalence class of R_L . That is, for any DFA $M = (Q, \Sigma, \delta, s, F)$ accepting L we define a new equivalence class R_L^* on Q as follows:

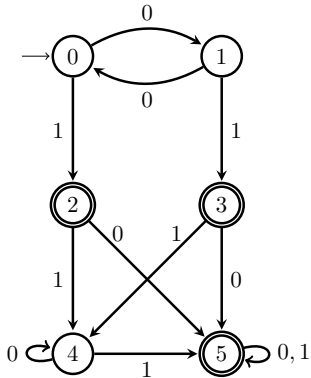
$$p R_L^* q \iff \forall \omega [\delta^*(p, \omega) \in F \leftrightarrow \delta^*(q, \omega) \in F].$$

From relation R_L^* , we can build the minimum DFA $M^* = (Q_m, \Sigma, \delta_m, s_m, F_m)$ for L as follows:

- $Q_m = \{[q]_{R_L^*} : q \in Q\};$
- $\delta_m([q]_{R_L^*}, a) = [\delta(q, a)]_{R_L^*};$
- $s_m = [s]_{R_L^*};$
- $F_m = \{[f]_{R_L^*} : f \in F\}.$

Example

Find the minimum DFA equivalent to the following DFA:



Example

Find the minimum DFA equivalent to the following DFA:

